

Lauda Local

Decisões técnicas

Arquitetura, escolhas de projeto, medições
e os limites que aceitamos de olhos abertos.

Versão 0.15.0-beta | 25/09/2026

Documento para quem mantém o código

Sumário

1. O problema e a forma da solução	3
O pipeline	3
Regra que atravessa todo o código	3
2. Decisões de plataforma	4
Motor de transcrição: faster-whisper (CTranslate2)	4
ffmpeg como dependência de sistema	4
Python 3.12	4
Interface: janela nativa em Tkinter	4
3. Decisões de qualidade da transcrição	5
VAD ligado por padrão	5
condition_on_previous_text desligado	5
Modo sequencial como padrão, lote como opção	5
A medição que decidiu isso	5
4. Escolha automática de hardware	6
AMD, Intel e a placa que não é usada	6
Núcleos físicos, não threads lógicas	6
Bibliotecas CUDA instaladas via pip	7
5. Confiabilidade: retomada e supervisão	8
Processo separado, não thread	8
Matar a árvore, não só o filho	8
Ponto de retomada por etapa	8
O que invalida um ponto salvo	8
6. Limites de recurso: o que é real e o que não seria	9
Os quatro controles viraram a segunda aba	9
7. Segurança da informação	10
Modelo de ameaça	10
O que o projeto NÃO protege	10
8. Desempenho: onde o tempo realmente vai	11
O que foi otimizado, então	11
Peso em disco	11

9. O que foi deixado de fora, de propósito	12
10. Qualidade: como sabemos que funciona	13
Os testes que mais importam	13
11. Empacotar para outra máquina	14
Ambiente de build separado, sem CUDA e sem Gradio	14
ffmpeg embutido, numa compilação LGPL	14
Uma pasta, não um arquivo único	14
Podar o que o torch traz e não usa	14
O que o usuário baixa	15
12. Limites conhecidos	16
Se este projeto continuar	16

1. O problema e a forma da solução

O objetivo é transformar um arquivo de áudio ou vídeo em um laudo de texto com tudo o que dá para extrair **sem enviar nada para lugar nenhum**. Essa restrição — processamento local — não é um detalhe de implantação: ela determina praticamente todas as decisões deste documento.

O pipeline

```
arquivo
|
+-- probe      ffprobe -> metadados (container, streams, tags)
+-- extract    ffmpeg  -> WAV PCM 16 bits, 16 kHz, mono
+-- vad        volume, silencio, clipping
+-- asr        faster-whisper -> trechos + idioma
+-- align      encolhe bordas pelas palavras, remove sobreposicao
+-- diarize    embeddings de locutor + clusterizacao (opcional)
+-- render     TXT, TXT corrido, JSON, SRT, VTT
```

Cada etapa é um módulo com uma responsabilidade só, e o `pipeline.py` apenas orquestra. Isso não é purismo: é o que permite salvar um ponto de retomada entre etapas e testar cada uma isoladamente.

Regra que atravessa todo o código

Etapas obrigatória falha alto; etapas opcional nunca derruba o trabalho. Sem o `ffprobe` não há o que fazer — o programa para com uma mensagem clara. Já a diarização, o resumo e a camada visual, se falharem, viram uma linha `[INDISPONÍVEL] <bloco>: <motivo>` no relatório. O leitor sempre sabe o que faltou e por quê; nada some em silêncio.

2. Decisões de plataforma

Motor de transcrição: faster-whisper (CTranslate2)

Escolha	faster-whisper 1.2, com os pesos abertos do Whisper.
Por quê	É a implementação que roda o Whisper mais rápido em CPU e GPU sem exigir PyTorch. Os pesos são MIT. Alternativas: a API da OpenAI está fora por definição (nuvem); o whisper.cpp seria ótimo em Apple Silicon, mas exigiria compilar por plataforma e não traz VAD e timestamps por palavra prontos.
O que custa	Depende de bibliotecas CUDA separadas para usar a GPU — a origem do erro mais comum do projeto (ver seção 8).

ffmpeg como dependência de sistema

Escolha	Chamar o binário `ffmpeg`/`ffprobe` instalado pelo usuário.
Por quê	Reimplementar leitura de container e decodificação seria absurdo, e empacotar o ffmpeg mudaria a licença do projeto (os builds completos são GPL). Chamando o binário instalado, a licença aplicável é a do build do usuário, e nós continuamos MIT.
O que custa	Uma dependência a instalar — mitigada com detecção automática em vários diretórios e mensagem de erro com o comando exato de instalação.

Python 3.12

Escolha	Faixa suportada 3.11–3.13; 3.12 é a recomendada.
Por quê	O 3.14 ainda não tem wheels de `ctranslate2`. Fixar a faixa evita que alguém instale no interpretador errado e receba um erro de compilação incompreensível.
O que custa	Não roda no Python mais novo. É temporário e está documentado.

Interface: janela nativa em Tkinter

Escolha	Tkinter, com a interface Gradio mantida como alternativa.
Por quê	O pedido foi explícito: um aplicativo, não uma página web local. O Tkinter vem com o Python — nenhuma dependência nova, nenhum servidor, nenhuma porta aberta — e dá acesso direto aos diálogos de arquivo do sistema. Electron ou PyQt trariam centenas de MB para resolver o mesmo problema.
O que custa	Visual menos moderno que o Gradio. Compensado usando o tema `clam` do ttk, que é o único totalmente recolorível — condição para o modo escuro.

3. Decisões de qualidade da transcrição

As três decisões abaixo existem por causa de um comportamento conhecido do Whisper: **em silêncio, ele inventa texto**. Não é bug, é como o modelo foi treinado.

VAD ligado por padrão

Escolha	Filtro de voz (Silero) ativo, com 500 ms de silêncio mínimo.
Por quê	Sem ele, trechos mudos viram frases inventadas — normalmente repetições de algo dito antes. O VAD corta o silêncio antes de o modelo ver.
O que custa	Um trecho de fala muito baixa pode ser descartado. O relatório avisa quando o volume médio está abaixo de -38 dB.

condition_on_previous_text desligado

Escolha	Cada janela é transcrita sem o texto anterior como contexto.
Por quê	Com o encadeamento ligado, uma alucinação contamina todas as janelas seguintes, e o resultado degradingola até o fim do arquivo.
O que custa	Perde-se um pouco de coerência de pontuação entre janelas. Vale a troca.

Modo sequencial como padrão, lote como opção

Escolha	<code>`batch_size = 0`</code> (sequencial) por padrão; o modo em lote é opt-in.
Por quê	Medimos o modo em lote: 2,0x mais rápido na GPU e 1,3x na CPU . Mas os trechos saem ~6x mais longos (média de 4,3 s contra 29,7 s). Um trecho de 30 segundos é inútil como legenda e atrapalha a diarização, que atribui um falante por trecho.
O que custa	Quem quer velocidade precisa ligar a opção e aceitar o texto menos granular — dito com essas palavras na própria tela.

A medição que decidiu isso

Cenário	Tempo	Trechos	Trecho médio
GPU, sequencial	12,54 s	60	4,3 s
GPU, em lote (16)	6,25 s	10	29,7 s
CPU, sequencial	30,83 s	62	4,9 s
CPU, em lote (8)	23,05 s	10	30,4 s

Áudio de 5 minutos, modelo small, GTX 1650 SUPER (int8_float16) e CPU de 12 threads (int8). O tempo mede a iteração completa do gerador — medir só a chamada daria números falsos, porque o trabalho acontece na iteração.

4. Escolha automática de hardware

O aplicativo nunca deve falhar por falta de memória, e nunca deve exigir que o usuário saiba o que é VRAM. A decisão é tomada em três camadas.

- **Antes de carregar** — calcula um orçamento de memória (VRAM menos 0,8 GB de reserva do runtime CUDA, ou uma fração da RAM na CPU) e escolhe o maior modelo que cabe. Se o pedido não couber, rebaixa e escreve o motivo no relatório.
- **Ao carregar** — se a GPU falhar (falta de biblioteca, driver antigo), recarrega na CPU automaticamente.
- **Durante a inferência** — o erro mais traiçoeiro aparece só na primeira inferência, não no carregamento. Nesse caso a transcrição inteira é refeita na CPU em vez de devolver um relatório vazio.

Por que a terceira camada existe

Na máquina de desenvolvimento, o modelo carregava na GPU sem erro e quebrava com `Library cublas64_12.dll is not found` na primeira inferência. A versão que só tratava falha no carregamento entregava um relatório sem transcrição nenhuma — e sem explicação decente.

AMD, Intel e a placa que não é usada

O CTranslate2 tem exatamente dois caminhos: `cpu` e `cuda`. Não há ROCm, não há DirectML, não há Metal — dá para conferir chamando `get_supported_compute_types`. Uma Radeon ou uma Intel, por mais nova que seja, não acelera a transcrição.

Esse fato não era o problema. O problema era o programa não saber diferenciar **três situações** que ele resumia todas como “nenhuma GPU CUDA detectada”: não ter placa, ter uma placa de outro fabricante, e ter a placa certa com o driver pela metade. Quem tinha uma RX 7800 XT lia aquilo como driver quebrado e ia atrás de um conserto que não existe; quem tinha uma GeForce com cuDNN faltando não descobria que o conserto existia.

Por isso o programa enumera os adaptadores pelo identificador do fabricante no barramento PCI — registro no Windows, `/sys` no Linux, sem abrir subprocesso. É a única fonte que continua certa quando o driver certo não está instalado. Com ela, cada situação ganha a frase que leva à ação certa, e só uma delas pede conserto.

O que faltaria para acelerar em AMD

Um segundo motor de inferência: `whisper.cpp` (que tem Vulkan e ROCm) ou ONNX Runtime com DirectML. Não é configuração, é outro modelo, outro formato de arquivo e outro caminho de código para manter. Enquanto isso não existir, dizer a verdade e tirar o melhor do processador é o melhor que dá para fazer — e é o que o programa faz.

Núcleos físicos, não threads lógicas

Quem não tem placa NVIDIA roda tudo no processador, e aí o número de threads passa a ser a decisão de desempenho mais importante. A intuição diz para usar todas. A medição diz o contrário: as duas threads de um mesmo núcleo disputam a unidade de cálculo que as multiplicações de matriz do modelo já saturam com uma thread só.

Threads	4 min de áudio, modelo small	x tempo real
12 (todas as lógicas)	29,9 s / 26,6 s	8,15x / 9,16x
6 (os núcleos físicos)	24,8 s / 24,2 s	9,84x / 10,07x

Duas medições de cada, num Ryzen 5 5600X de 6 núcleos e 12 threads, com o mesmo áudio e a mesma saída (424 palavras nas quatro rodadas): **~13% mais rápido usando metade das threads**. Por isso o controle de CPU da página Desempenho conta núcleos físicos, e 100% significa 6 aqui, não 12.

Bibliotecas CUDA instaladas via pip

O CTranslate2 procura cuBLAS e cuDNN no PATH do processo. Quando elas vêm dos pacotes `nvidia-*-cu12`, ficam em `site-packages/nvidia/*/bin`, que não está no PATH. O aplicativo registra esses diretórios com `os.add_dll_directory` antes do primeiro uso da GPU. Sem isso, a instalação por pip simplesmente não funciona — e o erro não diz o porquê.

5. Confiabilidade: retomada e supervisão

Processo separado, não thread

Escolha	O trabalho roda num processo filho supervisionado pelo pai.
Por quê	Uma thread travada dentro de uma chamada nativa do CTranslate2 não pode ser morta — não há sinal que a interrompa. Um processo, sim. Sem essa separação, o recurso de recuperação seria decorativo.
O que custa	Custo de arranque de um processo por trabalho e um protocolo simples (uma linha JSON por evento) entre pai e filho.

Matar a árvore, não só o filho

Escolha	<code>taskkill /F /T</code> no Windows; grupo de processos no POSIX.
Por quê	O filho chama o ffmpeg. Encerrar só o filho deixava o ffmpeg vivo, comendo CPU e segurando o arquivo aberto — foi o que aconteceu no primeiro teste manual, e por isso existe um teste automatizado que cria um processo neto e confirma que ele morre junto.
O que custa	Nenhum relevante.

Ponto de retomada por etapa

Escolha	Estado gravado depois de extract, vad, asr, align e diarize.
Por quê	A transcrição é 95% do tempo. Perdê-la porque a diarização quebrou seria inaceitável. Com o ponto salvo, a segunda tentativa aproveita tudo o que já estava pronto — medimos 28,3 s até a morte e 14,4 s na retomada.
O que custa	Ocupa espaço em disco (o WAV extraído). Mitigado extraindo direto para a pasta do ponto, sem cópia, e apagando tudo quando o trabalho conclui.

O que invalida um ponto salvo

A chave é o SHA-256 do arquivo mais uma impressão digital das opções que mudam o **conteúdo** do trabalho pesado:

Muda o ponto salvo	Não muda
modelo, idioma, device efetivo, compute_type, beam_size, batch_size, VAD, timestamps por palavra, diarização e seus parâmetros	pasta de saída, formatos de legenda, JSON sim/não, camada visual, resumo, número de palavras frequentes

Trocar a pasta de saída não justifica transcrever tudo de novo; trocar o modelo, sim. Essa distinção é o que torna a retomada útil na prática. E, entre tentativas, repetir a configuração que acabou de travar teria pouca chance: por isso a segunda desliga a GPU e a terceira reduz o modelo, anunciando cada degrau ao usuário.

6. Limites de recurso: o que é real e o que não seria

Quatro controles deslizantes. A tentação era fazer os quatro parecerem iguais; a decisão foi mapear cada um em algo que **de fato existe** e dizer na tela o que ele faz.

Controle	Mecanismo real	É um limite rígido?
CPU	Número de threads do CTranslate2 e do ffmpeg; abaixo de 60% também rebaixa a prioridade do processo.	Sim — é contagem de núcleos.
RAM	Teto do orçamento usado para escolher o modelo em CPU.	Não — é um limite de escolha, não um `ulimit`.
VRAM	Mesmo mecanismo, aplicado à memória da placa.	Não — o CTranslate2 não expõe teto de alocação.
GPU	0% ignora a placa; acima disso ajusta o paralelismo (`num_workers`).	Parcial — ligar/desligar é rígido; a porcentagem é grosseira.

A alternativa que foi recusada

Seria fácil mostrar um controle de 0 a 100% de GPU sugerindo que ele limita a ocupação da placa. Não existe essa API no CUDA. Um controle que não corresponde a nada é pior do que não ter controle: o usuário toma decisões erradas confiando nele.

Os quatro controles viraram a segunda aba

A tabela acima é honesta e continua valendo — e era ilegível para quem não sabe o que é uma thread. A página Desempenho passou a ter três abas: **Resumo** (quatro opções prontas, em português comum), **Limites** (estes controles) e **Diagnóstico** (o que foi medido, sempre na tela). Os presets não são enfeite: cada um é um conjunto de valores destes mesmos quatro controles, e mexer num controle apaga o destaque do preset — porque aí a configuração deixou de ser aquela.

O Resumo também compara o trabalho pedido com o que os limites liberam, e avisa **antes** quando o modelo vai ser rebaixado por falta de memória. Era informação que só existia depois, dentro do laudo, quando já se tinha esperado a transcrição inteira.

7. Segurança da informação

Modelo de ameaça

O aplicativo roda com os privilégios do usuário, na máquina dele, sobre arquivos dele. Não há autenticação, multiusuário nem superfície de rede a defender. As ameaças que sobram são três: **vazamento por descuido** (dado sensível indo parar onde não devia), **entrada maliciosa** (um arquivo de mídia preparado) e **configuração hostil** (variáveis de ambiente apontando para lugares inesperados).

Medida	Contra o quê
Nenhum envio de mídia pela rede. O único acesso externo é o download inicial dos modelos, e existe modo estritamente offline.	Vazamento do conteúdo processado.
O arquivo de mídia nunca é executado: é passado como argumento de leitura para o ffmpeg, sempre por lista de argumentos, nunca por shell.	Injeção de comando por nome de arquivo, execução de conteúdo.
O token do Hugging Face é removido antes de serializar as opções para o processo filho; ele viaja pelo ambiente, que não deixa rastro em disco.	Segredo em arquivo temporário legível por outros usuários.
Pontos de retomada guardam transcrições: a pasta é criada com 0700 no POSIX e herda a ACL do perfil no Windows. São apagados ao concluir e expiram em 7 dias.	Conteúdo sensível esquecido no disco.
O endereço do Ollama é validado: só `http` e `https` passam.	`OLLAMA_HOST=file:///...` fazendo o cliente ler arquivos locais.
Nomes de arquivo de saída passam por normalização que remove acentos, separadores e caracteres de caminho.	Travessia de diretório a partir do nome do arquivo de entrada.
Toda saída do ffmpeg é tratada como dado não confiável: conversões numéricas defensivas, nada de `eval`.	Metadados manipulados quebrando o programa.

O que o projeto NÃO protege

- Não há criptografia em repouso. Quem tem acesso ao perfil do usuário lê os relatórios e os pontos de retomada.
- Não há sandbox para o ffmpeg. Uma vulnerabilidade no ffmpeg é uma vulnerabilidade aqui — mantenha-o atualizado.
- A interface no navegador com `--share` escuta em todas as interfaces, sem autenticação. É intencional e serve para rede local confiável; a interface padrão (a janela nativa) não abre porta nenhuma.
- Os modelos baixados não têm verificação de assinatura além do que o Hugging Face já faz.

8. Desempenho: onde o tempo realmente vai

Antes de otimizar qualquer coisa, medimos. O resultado desmontou a lista de otimizações que parecia óbvia.

Operação	Tempo (áudio de 5 min)	Vale otimizar?
SHA-256 do arquivo	6 ms	só se for calculado duas vezes
ffprobe	44 ms	não
extração para WAV	72 ms	não
análise de volume/silêncio	80 ms	não
transcrição (GPU, small)	12 540 ms	é aqui que está tudo

Juntar as passagens do ffmpeg numa só economizaria dezenas de milissegundos num trabalho de dezenas de segundos. Foi descartado: complicaria o código para um ganho invisível.

O que foi otimizado, então

- **SHA-256 calculado uma vez.** O supervisor precisa do hash para achar o ponto de retomada e o pipeline precisa dele para a identidade do arquivo. Eram duas leituras completas — num vídeo de 4 GB, alguns segundos por execução, à toa.
- **WAV extraído direto na pasta do ponto de retomada.** Antes ele era extraído no temporário e copiado; a cópia podia ter centenas de MB.
- **Modo em lote disponível** para quem prioriza velocidade, com o custo declarado (seção 3).

Peso em disco

Uma instalação completa ocupa cerca de 3,2 GB de bibliotecas — e **61% disso são as bibliotecas CUDA**, que só existem para acelerar na placa de vídeo. Como ninguém descobre isso sozinho, o programa passou a mostrar a conta e a permitir liberar espaço:

```
lauda disco
```

Componente	Tamanho	Necessário?
aceleração por GPU (CUDA)	1,9 GB	não, se rodar em CPU
diarização (PyTorch e cia.)	726 MB	não, se não separar falantes
núcleo da transcrição	236 MB	sim
interface no navegador	148 MB	não, o app nativo dispensa
geração dos PDFs	34 MB	não, só para regerar os manuais
modelos baixados	varia	baixa de novo quando precisar

Uma instalação só de CPU, sem diarização e sem interface web, fica em torno de **400 MB** — oito vezes menor.

9. O que foi deixado de fora, de propósito

Recurso	Por que não
Alinhamento forçado (wav2vec2, estilo WhisperX)	Mais ~1 GB de modelo por idioma para um ganho pequeno sobre os timestamps por palavra que o Whisper já entrega. O que fazemos em lugar disso: encolher as bordas do trecho até a primeira e a última palavra, e remover sobreposições.
Descrição visual do vídeo com um modelo de visão	Custa GB de download e minutos por vídeo, para um app cujo produto é texto falado. O gancho existe em `visual.py`, desligado.
Tradução	O Whisper traduz apenas para o inglês, e mal. Prometer tradução seria prometer qualidade que o modelo não entrega.
Formatador automático de código (`ruff format`)	Reformataria as tabelas de dados do relatório e dos manuais para uma linha por item, tornando-as bem menos legíveis. O lint está ativo e limpo; a formatação é mantida à mão.
Cortar o áudio em janelas antes de transcrever	Pedido no backlog (janela deslizante de 10 a 60 s, chunk duplo). O faster-whisper já percorre o arquivo inteiro em fluxo; fatiar o áudio acrescentaria emendas — e é exatamente na emenda que texto some, que foi o defeito relatado no BACKLOG-006. A parte aproveitável da ideia, cortar onde a fala respira , foi aplicada à legenda (`cues.py`), não ao áudio.
Empacotar um ffmpeg GPL	O que vai junto é uma compilação LGPL (BtbN). As builds completas, com x264 e afins, são GPL e obrigariam o projeto inteiro a virar GPL — por transcrever, que não usa nada disso.

10. Qualidade: como sabemos que funciona

Ferramenta	Estado	O que cobre
pytest	151 testes, todos passando	Do formato do timestamp ao ciclo completo de travar-matar-retomar.
ruff	sem apontamentos	Erros prováveis, segurança (regras bandit), imports, modernização.
mypy	sem apontamentos	30 arquivos, com <code>check_untyped_defs</code> e sem <code>type: ignore</code> sobrando.

Os testes que mais importam

- **Determinismo** — o mesmo arquivo com as mesmas opções produz um relatório com layout idêntico; só variam relógio e tempos medidos.
- **Blocos indisponíveis** — um vídeo mudo gera relatório completo com [INDISPONÍVEL] Transcrição, não uma exceção.
- **Retomada sem retrabalho** — com um ponto salvo na etapa `asr`, o teste substitui a função de transcrição por uma que falha se for chamada. Se a retomada não funcionasse, o teste quebraria.
- **Processo neto órfão** — o worker falso cria um neto que escreve num arquivo a cada 0,2 s. Depois do travamento e da matança, o arquivo tem de parar de crescer.
- **Segredo em disco** — verifica que o token não aparece no JSON que o supervisor grava para o processo filho.
- **Isolamento do perfil** — uma fixture obrigatória redireciona `~/ .lauda` para uma pasta temporária. Ela foi criada depois que um teste de verdade escreveu no perfil real.

Sobre testes que não testam nada

Vários dos testes acima existem porque um bug apareceu primeiro: o supervisor que nunca detectava travamento na transcrição, o ffmpeg órfão, o perfil real sendo sujo pelos testes. Teste que não nasce de um problema concreto costuma só repetir a implementação.

11. Empacotar para outra máquina

Sair do ambiente de desenvolvimento e virar um instalador que roda na máquina de outra pessoa levantou quatro decisões — nenhuma delas óbvia, todas com um custo escrito.

Ambiente de build separado, sem CUDA e sem Gradio

Escolha	Um segundo venv (.venv-build) instala o torch pelo índice CPU do PyTorch e não instala o extra da interface web.
Por quê	As bibliotecas CUDA sozinhas somam 1,7 GB — 61% do venv de desenvolvimento — e o aplicativo de janela não usa Gradio. Empacotar o ambiente de desenvolvimento levaria o download de 227 MB para mais de 2 GB.
O que custa	Quem tem placa NVIDIA precisa instalar o extra `gpu` depois, à mão. O script de build recusa rodar se encontrar esses pacotes, para o engano não passar despercebido.

ffmpeg embutido, numa compilação LGPL

Escolha	O pacote leva ffmpeg e ffprobe junto, de uma compilação LGPL v3.
Por quê	As compilações comuns do ffmpeg são GPL: distribuí-las junto obrigaria o projeto inteiro, hoje MIT, a virar GPL. A LGPL permite embutir ao lado de um aplicativo de outra licença, com aviso e ponteiro para o código-fonte. Em troca, o usuário deixa de precisar instalar o ffmpeg à parte — que era o passo onde mais gente travava.
O que custa	Uns 120 MB e a obrigação de manter o aviso de licença junto. Os codecs GPL (x264, x265) não vêm — o aplicativo não usa nenhum deles.

Uma pasta, não um arquivo único

Escolha	O PyInstaller gera um diretório com o executável e as bibliotecas ao lado, não um --onefile.
Por quê	O modo de arquivo único descompacta ~900 MB num temporário a cada abertura: dezenas de segundos de espera e um alvo fácil para o antivírus. Em pasta, o programa abre na hora.
O que custa	A distribuição precisa de um instalador para não virar “descompacte isto em algum lugar”. Daí o Inno Setup.

Podar o que o torch traz e não usa

Escolha	Os cabeçalhos C++ (torch/include), as bibliotecas de link e as licenças de terceiros saem do pacote.
Por quê	São ~90 MB que só servem para compilar extensões. E há um motivo mais concreto: eles criam caminhos como <code>predicated_tile_access_iterator_residual_last.h</code> , que estouram o limite de 260 caracteres do Windows e fazem a instalação falhar em máquina com nome de usuário comprido. Foi assim que o problema apareceu.

O que custa

Se um dia o projeto precisar compilar uma extensão do torch em tempo de execução, a poda tem de ser revista.

O que o usuário baixa

Item	Tamanho	Observação
Instalador	~227 MB	É o que você manda pela internet.
Instalado em disco	~860 MB	Programa, bibliotecas e ffmpeg.
Modelo de transcrição	0,5 a 3 GB	Baixado na primeira execução, uma vez. Fica em ~/.lauda — desinstalar não apaga.

O executável não é assinado

Uma assinatura de código custa algumas centenas de dólares por ano. Sem ela, o Windows mostra a tela do SmartScreen na primeira execução. O texto de **ANTES-DE-INSTALAR.txt** explica isso ao usuário antes que ele se assuste — omitir o aviso é o caminho mais curto para alguém desistir da instalação achando que baixou um vírus.

12. Limites conhecidos

Nenhum destes é um defeito a corrigir depois: são características assumidas, e o programa fala sobre elas onde o usuário vai encontrá-las.

- **O Whisper erra** com sotaque forte, ruído, música e falas sobrepostas. Números, siglas e nomes próprios são o que mais escapa. A opção de prompt inicial existe justamente para ancorar vocabulário.
- **Os timestamps por palavra são estimativas** do decodificador, não medição acústica. Servem para legendar, não para perícia.
- **A diarização só rotula o que foi transcrito.** Fala sobreposta vira um falante só; vozes parecidas podem ser fundidas.
- **A qualidade depende mais do áudio que do modelo.** Microfone perto e ambiente silencioso melhoram mais que trocar `small` por `large`.
- **Não substitui revisão humana** em uso jurídico, jornalístico, médico ou acadêmico.

Se este projeto continuar

Na ordem em que fariam mais diferença: processamento de uma pasta inteira em fila; usar as probabilidades por palavra para marcar no relatório os trechos de baixa confiança (onde revisar primeiro); e o executor completo do pyannote, hoje disponível mas sem o mesmo cuidado de teste que o backend ECAPA recebeu.